

Web Development Assignment – 5

Topic:

**Weather Forecast Application using React.js, Node.js,
GraphQL, REST API, OAuth & WebSocket**

Weather Forecast System:

Student Details

Name: Esha Patel

Enrollment No.: 230090107130

College: C.K Pithawala college of engineering & Technology

Technology: React.js, Node.js, Express.js, REST API, GraphQL, Socket.IO
(WebSocket), Google OAuth, WeatherAPI, Axios

Objective

The objective of this assignment is to develop a Weather Forecast Application that demonstrates the integration of multiple modern web technologies including REST APIs, GraphQL, OAuth Authentication, and WebSocket communication.

Introduction

The Weather Forecast Application allows users to search weather information for any city around the world. The application communicates with WeatherAPI through a Node.js backend and displays weather information using React.js.

The project also includes:

- REST API Development
- GraphQL API
- Google OAuth Login
- WebSocket Communication using Socket.IO
- API Documentation
- GraphQL Schema Documentation

Software Requirements

Frontend

- React.js
- Axios
- React Router
- Socket.IO Client

Backend

- Node.js
- Express.js
- GraphQL
- Express GraphQL
- Passport.js

- Google OAuth 2.0
- Socket.IO
- dotenv

GraphQL API

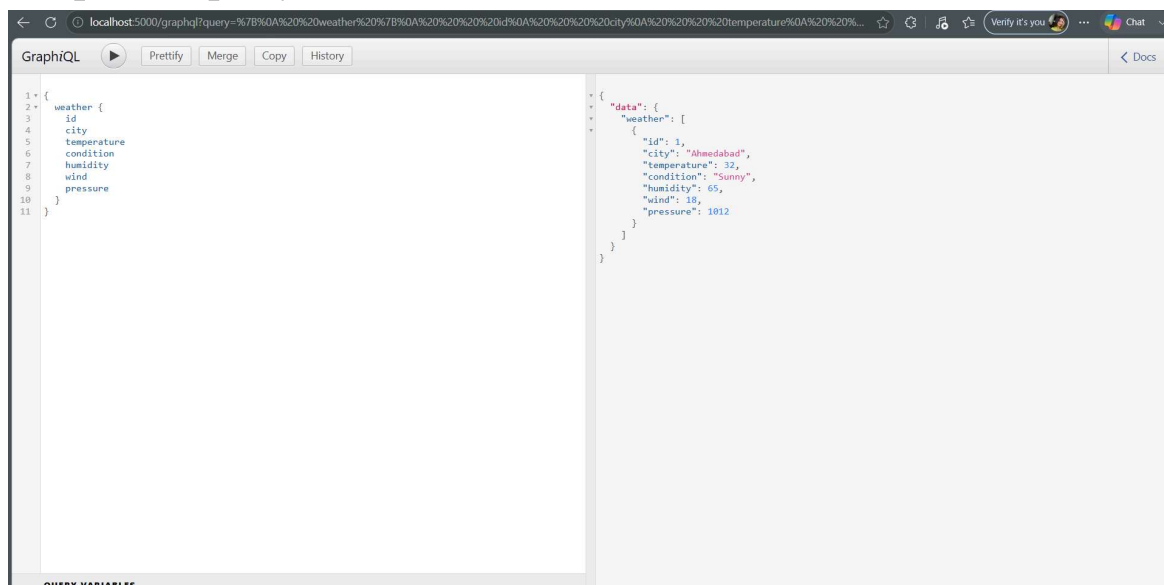
GraphQL Endpoint

<http://localhost:5000/graphql>

Example Query

```
{
  weather {
    id
    city
    temperature
    condition
    humidity
    wind
    pressure
  }
}
```

GraphiQL query/result screenshot



GraphQL Schema Documentation

User Type

```
type User{
  id:Int
  name:String
  email:String
}
```

Query

```
type Query{
  users:[User]
  weather(city:String!):Weather
}
```

Weather Type

```
type Weather{
  location:Location
  current:Current
}
```

Location Type

```
type Location{
  name:String
  country:String
}
```

Current Type

```
type Current{
  temp_c:Float
  humidity:Int
  wind_kph:Float
}
```

Google OAuth Authentication

Implemented using:

- Passport.js
- Google OAuth Strategy

WebSocket

Implemented using Socket.IO.

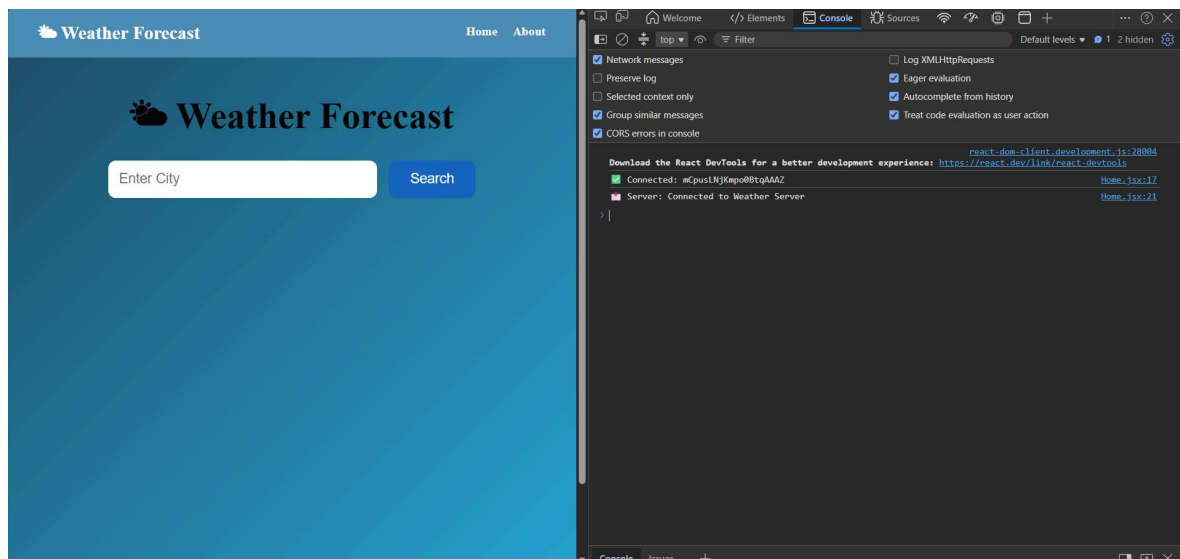
Server emits

```
socket.emit("message","Connected to Weather Server");
```

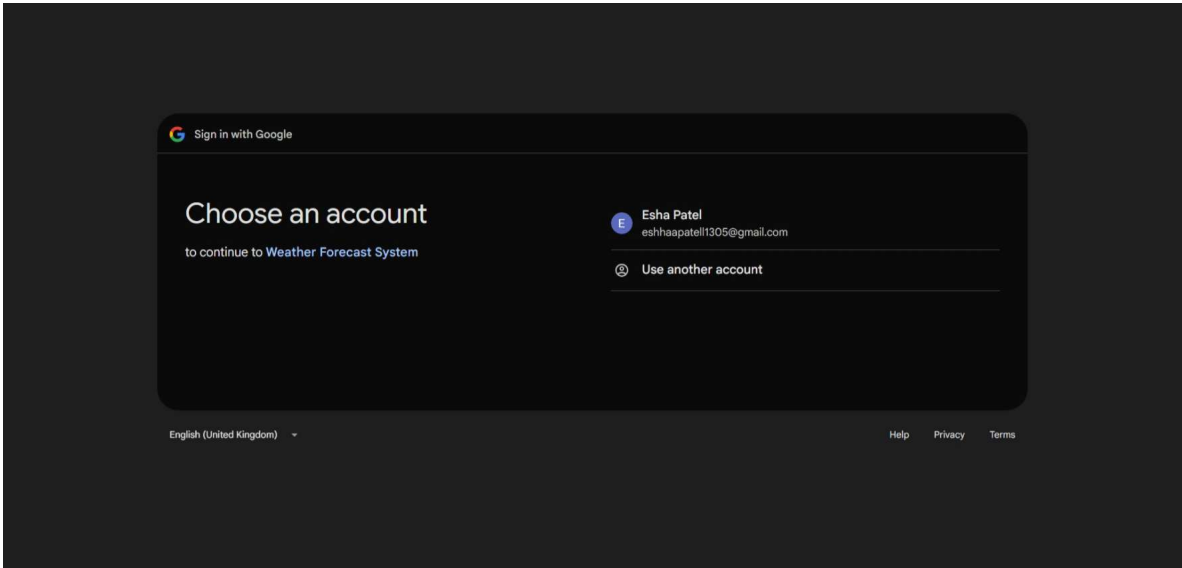
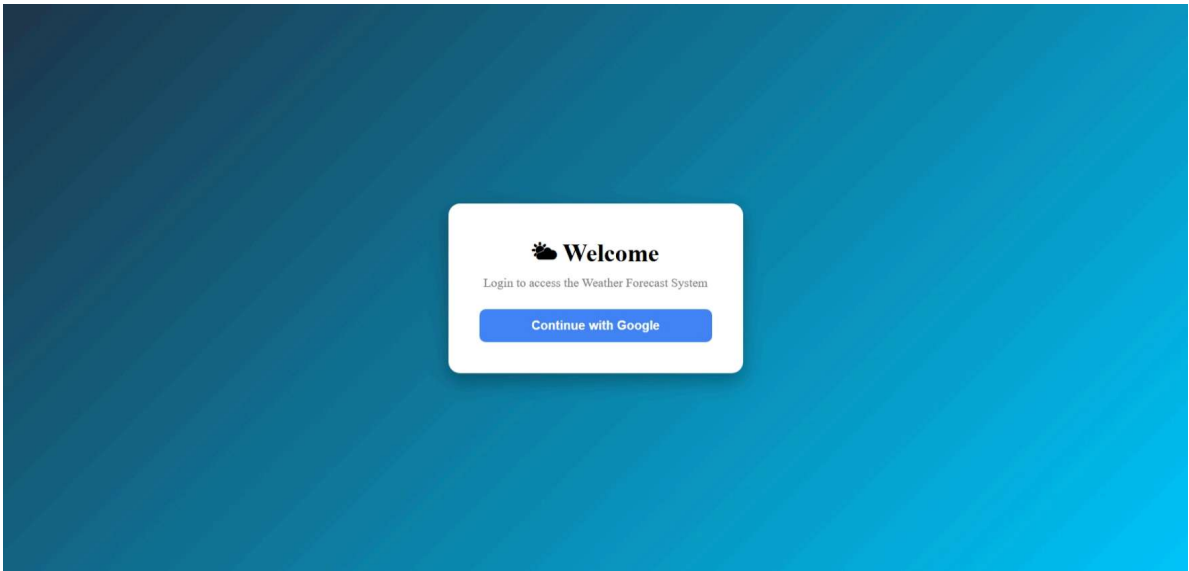
Client receives

```
socket.on("message",(msg)=>{  
  console.log(msg);  
});
```

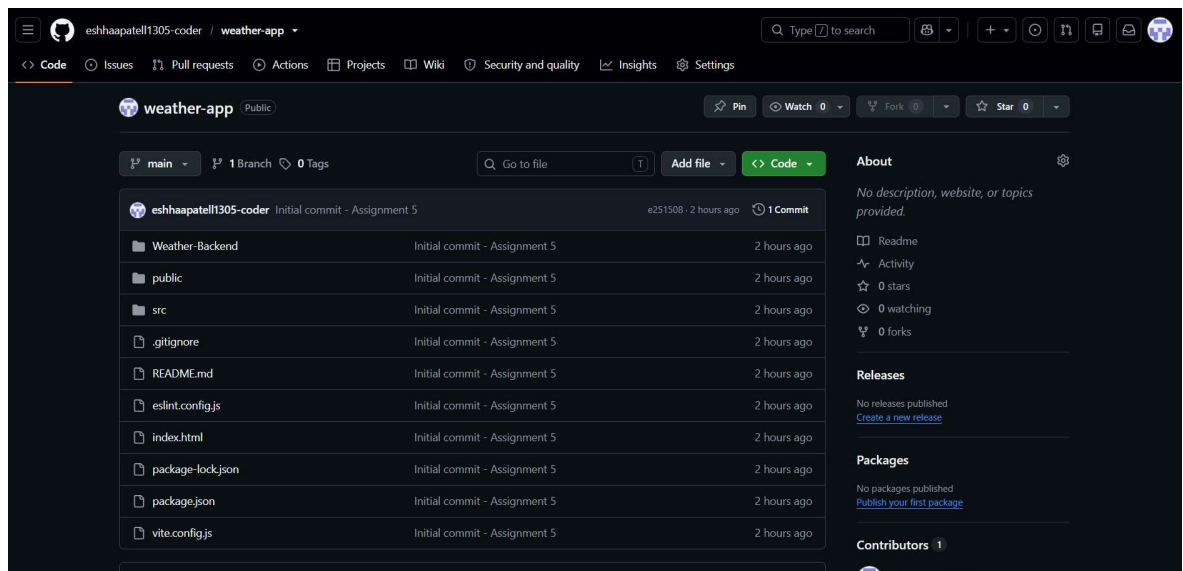
WebSocket console screenshot



Login Page with Google OAuth Authentication



GitHub repository screenshot



Learning Outcomes

Through this assignment, I learned:

- Building REST APIs using Express.js
- Creating GraphQL APIs
- Using GraphQL Playground
- Implementing Google OAuth Authentication
- Real-time communication using WebSocket
- Integrating React with Node.js
- API Testing and Documentation
- Version control using Git and GitHub

Challenges Faced

- Configuring Google OAuth credentials
- Resolving GitHub push protection due to exposed `.env` secrets
- Establishing Socket.IO connection between frontend and backend
- Integrating GraphQL with Express
- Managing frontend and backend simultaneously

Conclusion

The Weather Forecast Application successfully demonstrates the integration of React.js with a Node.js backend using multiple communication techniques such as REST APIs, GraphQL, OAuth authentication, and WebSockets. The project provided hands-on experience with modern full-stack development practices and strengthened understanding of API integration, authentication, real-time communication, and version control.